

Inigo Quilez :: articles :: 2D distance functions



Intro

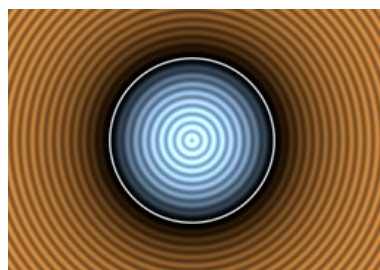
The [3D SDF functions article](#) is pretty popular, so I decided to write a similar one for 2D primitives, since most of the 3D primitives are grown as extrusions or revolutions of these 2D shapes. So getting these right is important. In particular, the functions bellow use the minimum number of square roots and divisions possible, and also produce better/faster results than constructing them from other primitives (when possible).

Note that all the primitives listed here come with a link to a realtime online demo in Shadertoy. In fact, this public playlist contains all the Shadertoy examples: <https://www.shadertoy.com/playlist/MXdSRf>, so don't miss that out.

Lastly, as with all in this website, all formulas and code are derived by myself (unless otherwise stated), so there might be errors or better ways to do things. Let me know if you think that's the case.

Primitives

All primitives are centered at the origin; you will have to transform the point to get arbitrarily rotated, translated and scaled objects (see below). The `dot2(v)` function returns the dot product of a vector with itself (or the square of its length).

**Circle - exact**

<https://www.shadertoy.com/view/3ltSW2>

```
float sdCircle( vec2 p, float r )
{
    return length(p) - r;
}
```

**Rounded Box - exact**

<https://www.shadertoy.com/view/4llXD7>

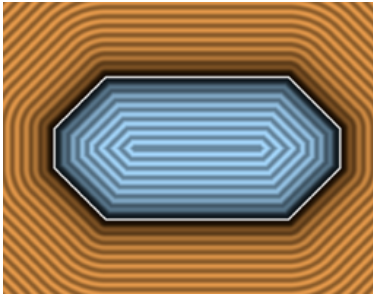
<https://www.youtube.com/watch?v=s5NGeUV2EyU>

```
float sdRoundedBox( in vec2 p, in vec2 b, in vec4 r )
{
    r.xy = (p.x>0.0)?r.xy : r.zw;
    r.x  = (p.y>0.0)?r.x  : r.y;
```

```

vec2 q = abs(p)-b+r.x;
return min(max(q.x,q.y),0.0) + length(max(q,0.0)) - r.x;
}

```



Chamfer Box - exact

<https://www.shadertoy.com/view/3fc3zs>

```

float sdChamferBox( in vec2 p, in vec2 b, in float chamfer )
{
    p = abs(p)-b;

    p = (p.y>p.x) ? p.yx : p.xy;
    p.y += chamfer;

    const float k = 1.0-sqrt(2.0);
    if( p.y<0.0 && p.y+p.x*k<0.0 )
        return p.x;

    if( p.x<p.y )
        return (p.x+p.y)*sqrt(0.5);

    return length(p);
}

```



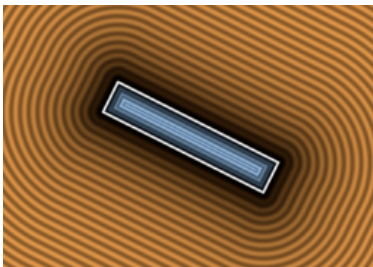
Box - exact

<https://www.youtube.com/watch?v=62-pRVZuS5c>

```

float sdBox( in vec2 p, in vec2 b )
{
    vec2 d = abs(p)-b;
    return length(max(d,0.0)) + min(max(d.x,d.y),0.0);
}

```



Oriented Box - exact

<https://www.shadertoy.com/view/stcfzn>

```

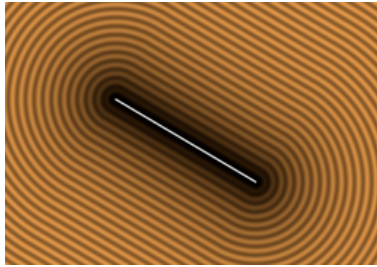
float sdOrientedBox( in vec2 p, in vec2 a, in vec2 b, float th )
{
    float l = length(b-a);

```

```

vec2 d = (b-a)/l;
vec2 q = (p-(a+b)*0.5);
q = mat2(d.x,-d.y,d.y,d.x)*q;
q = abs(q)-vec2(1,1)*0.5;
return length(max(q,0.0)) + min(max(q.x,q.y),0.0);
}

```



Segment - exact

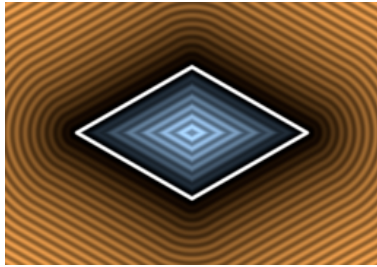
<https://www.shadertoy.com/view/3tdSDj>

<https://www.youtube.com/watch?v=PMltMdi1Wzg>

```

float sdSegment( in vec2 p, in vec2 a, in vec2 b )
{
    vec2 pa = p-a, ba = b-a;
    float h = clamp( dot(pa,ba)/dot(ba,ba), 0.0, 1.0 );
    return length( pa - ba*h );
}

```



Rhombus - exact

<https://www.shadertoy.com/view/XdXcRB>

```

float sdRhombus( in vec2 p, in vec2 b )
{
    b.y = -b.y;
    p = abs(p);
    float h = clamp( (dot(b,p)+b.y*b.y)/dot(b,b), 0.0, 1.0 );
    p -= b*vec2(h,h-1.0);
    return length(p)*sign(p.x);
}

```



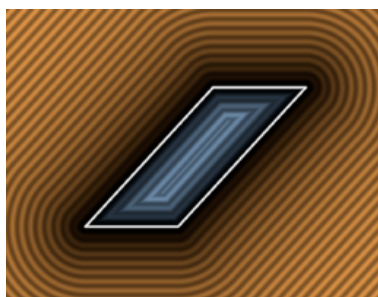
Isosceles Trapezoid - exact

<https://www.shadertoy.com/view/MlycD3>

```

float sdTrapezoid( in vec2 p, in float r1, float r2, float he )
{
    vec2 k1 = vec2(r2,he);
    vec2 k2 = vec2(r2-r1,2.0*he);
    p.x = abs(p.x);
    vec2 ca = vec2(p.x-min(p.x,(p.y<0.0)?r1:r2), abs(p.y)-he);
    vec2 cb = p - k1 + k2*clamp( dot(k1-p,k2)/dot2(k2), 0.0, 1.0 );
    float s = (cb.x<0.0 && ca.y<0.0) ? -1.0 : 1.0;
    return s*sqrt( min(dot2(ca),dot2(cb)) );
}

```



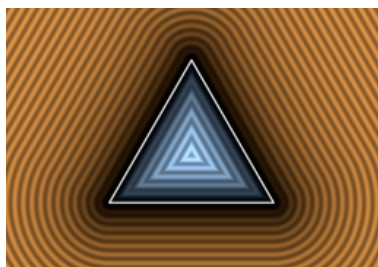
Parallelogram - exact

<https://www.shadertoy.com/view/7dIGRf>

```

float sdParallelogram( in vec2 p, float wi, float he, float sk )
{
    vec2 e = vec2(sk,he);
    p = (p.y<0.0)?-p:p;
    vec2 w = p - e; w.x -= clamp(w.x,-wi,wi);
    vec2 d = vec2(dot(w,w), -w.y);
    float s = p.x*e.y - p.y*e.x;
    p = (s<0.0)?-p:p;
    vec2 v = p - vec2(wi,0); v -= e*clamp(dot(v,e)/dot(e,e),-1.0,1.0);
    d = min( d, vec2(dot(v,v), wi*he-abs(s)));
    return sqrt(d.x)*sign(-d.y);
}

```



Equilateral Triangle - exact

<https://www.shadertoy.com/view/XI2yDW>

```

float sdEquilateralTriangle( in vec2 p, in float r )
{
    const float k = sqrt(3.0);
    p.x = abs(p.x) - r;
    p.y = p.y + r/k;
    if( p.x+k*p.y>0.0 ) p = vec2(p.x-k*p.y,-k*p.x-p.y)/2.0;
}

```

```

p.x -= clamp( p.x, -2.0*r, 0.0 );
return -length(p)*sign(p.y);
}

```



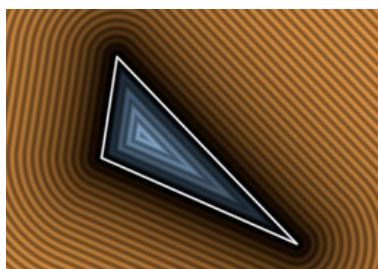
Isosceles Triangle - exact

<https://www.shadertoy.com/view/MldcD7>

```

float sdTriangleIsosceles( in vec2 p, in vec2 q )
{
    p.x = abs(p.x);
    vec2 a = p - q*clamp( dot(p,q)/dot(q,q), 0.0, 1.0 );
    vec2 b = p - q*vec2( clamp( p.x/q.x, 0.0, 1.0 ), 1.0 );
    float s = -sign( q.y );
    vec2 d = min( vec2( dot(a,a), s*(p.x*q.y-p.y*q.x) ),
                  vec2( dot(b,b), s*(p.y-q.y) ) );
    return -sqrt(d.x)*sign(d.y);
}

```



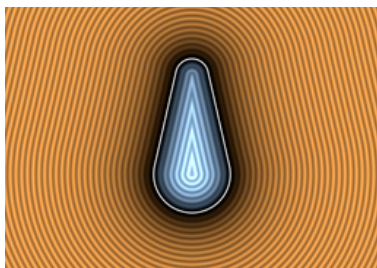
Triangle - exact

<https://www.shadertoy.com/view/XsXSz4>

```

float sdTriangle( in vec2 p, in vec2 p0, in vec2 p1, in vec2 p2 )
{
    vec2 e0 = p1-p0, e1 = p2-p1, e2 = p0-p2;
    vec2 v0 = p -p0, v1 = p -p1, v2 = p -p2;
    vec2 pq0 = v0 - e0*clamp( dot(v0,e0)/dot(e0,e0), 0.0, 1.0 );
    vec2 pq1 = v1 - e1*clamp( dot(v1,e1)/dot(e1,e1), 0.0, 1.0 );
    vec2 pq2 = v2 - e2*clamp( dot(v2,e2)/dot(e2,e2), 0.0, 1.0 );
    float s = sign( e0.x*e2.y - e0.y*e2.x );
    vec2 d = min(min(vec2(dot(pq0,pq0), s*(v0.x*e0.y-v0.y*e0.x)),
                    vec2(dot(pq1,pq1), s*(v1.x*e1.y-v1.y*e1.x))),
                vec2(dot(pq2,pq2), s*(v2.x*e2.y-v2.y*e2.x)));
    return -sqrt(d.x)*sign(d.y);
}

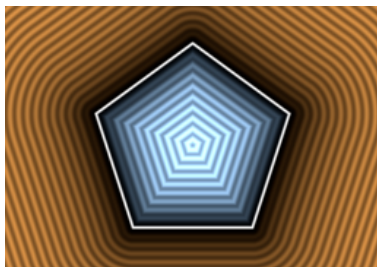
```



Uneven Capsule - exact

<https://www.shadertoy.com/view/4lcBWn>

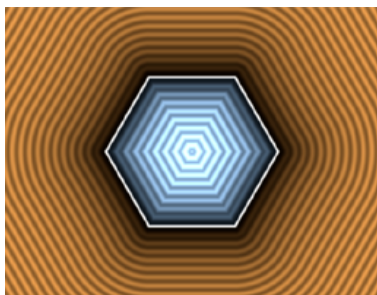
```
float sdUnevenCapsule( vec2 p, float r1, float r2, float h )
{
    p.x = abs(p.x);
    float b = (r1-r2)/h;
    float a = sqrt(1.0-b*b);
    float k = dot(p,vec2(-b,a));
    if( k < 0.0 ) return length(p) - r1;
    if( k > a*h ) return length(p-vec2(0.0,h)) - r2;
    return dot(p, vec2(a,b) ) - r1;
}
```



Regular Pentagon - exact

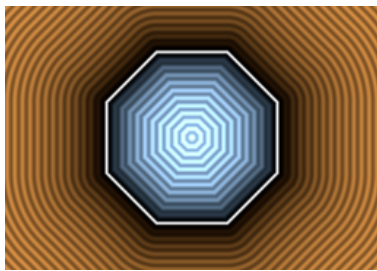
<https://www.shadertoy.com/view/llVyWW>

```
float sdPentagon( in vec2 p, in float r )
{
    const vec3 k = vec3(0.809016994,0.587785252,0.726542528);
    p.x = abs(p.x);
    p -= 2.0*min(dot(vec2(-k.x,k.y),p),0.0)*vec2(-k.x,k.y);
    p -= 2.0*min(dot(vec2( k.x,k.y),p),0.0)*vec2( k.x,k.y);
    p -= vec2(clamp(p.x,-r*k.z,r*k.z),r);
    return length(p)*sign(p.y);
}
```



Regular Hexagon - exact

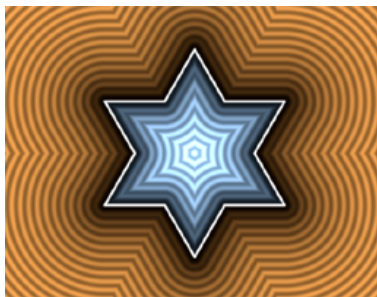
```
float sdHexagon( in vec2 p, in float r )
{
    const vec3 k = vec3(-0.866025404,0.5,0.577350269);
    p = abs(p);
    p -= 2.0*min(dot(k.xy,p),0.0)*k.xy;
    p -= vec2(clamp(p.x, -k.z*r, k.z*r), r);
    return length(p)*sign(p.y);
}
```



Regular Octagon - exact

<https://www.shadertoy.com/view/llGfDG>

```
float sdOctagon( in vec2 p, in float r )
{
    const vec3 k = vec3(-0.9238795325, 0.3826834323, 0.4142135623 );
    p = abs(p);
    p -= 2.0*min(dot(vec2( k.x,k.y),p),0.0)*vec2( k.x,k.y);
    p -= 2.0*min(dot(vec2(-k.x,k.y),p),0.0)*vec2(-k.x,k.y);
    p -= vec2(clamp(p.x, -k.z*r, k.z*r), r);
    return length(p)*sign(p.y);
}
```



Hexagram - exact

<https://www.shadertoy.com/view/tt23RR>

```
float sdHexagram( in vec2 p, in float r )
{
    const vec4 k = vec4(-0.5,0.8660254038,0.5773502692,1.7320508076);
    p = abs(p);
    p -= 2.0*min(dot(k.xy,p),0.0)*k.xy;
    p -= 2.0*min(dot(k.yx,p),0.0)*k.yx;
    p -= vec2(clamp(p.x,r*k.z,r*k.w),r);
    return length(p)*sign(p.y);
}
```



Pentagram - exact

<https://www.shadertoy.com/view/t3X3z4>

```
float sdPentagram(in vec2 p, in float r )
{
    const float k1x = 0.809016994; // cos( $\pi/5$ ) =  $\frac{1}{4}(\sqrt{5}+1)$ 
    const float k2x = 0.309016994; // sin( $\pi/10$ ) =  $\frac{1}{4}(\sqrt{5}-1)$ 
    const float k1y = 0.587785252; // sin( $\pi/5$ ) =  $\frac{1}{4}\sqrt{(10-2\sqrt{5})}$ 
    const float k2y = 0.951056516; // cos( $\pi/10$ ) =  $\frac{1}{4}\sqrt{(10+2\sqrt{5})}$ 
    const float k1z = 0.726542528; // tan( $\pi/5$ ) =  $\sqrt{(5-2\sqrt{5})}$ 
    const vec2 v1 = vec2( k1x,-k1y);
    const vec2 v2 = vec2(-k1x,-k1y);
    const vec2 v3 = vec2( k2x,-k2y);

    p.x = abs(p.x);
    p -= 2.0*max(dot(v1,p),0.0)*v1;
    p -= 2.0*max(dot(v2,p),0.0)*v2;
    p.x = abs(p.x);
    p.y -= r;
    return length(p-v3*clamp(dot(p,v3),0.0,k1z*r))
        * sign(p.y*v3.x-p.x*v3.y);
}
```

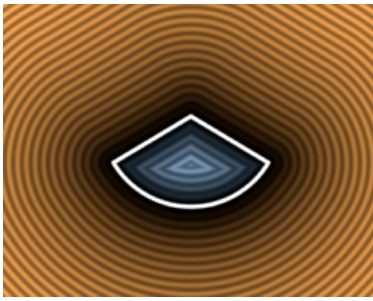


Regular Star - exact

<https://www.shadertoy.com/view/3tSGDy>

```
float sdStar( in vec2 p, in float r, in int n, in float m)
{
    // next 4 lines can be precomputed for a given shape
    float an = 3.141593/float(n);
    float en = 3.141593/m; // m is between 2 and n
    vec2 acs = vec2(cos(an),sin(an));
    vec2 ecs = vec2(cos(en),sin(en)); // ecs=vec2(0,1) for regular polygon

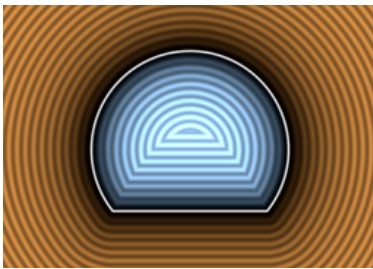
    float bn = mod(atan(p.x,p.y),2.0*an) - an;
    p = length(p)*vec2(cos(bn),abs(sin(bn)));
    p -= r*acs;
    p += ecs*clamp(-dot(p,ecs), 0.0, r*acs.y/ecs.y);
    return length(p)*sign(p.x);
}
```



Pie - exact

<https://www.shadertoy.com/view/3l23RK>

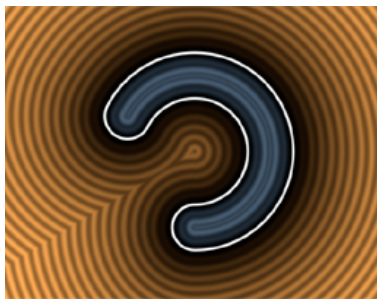
```
float sdPie( in vec2 p, in vec2 c, in float r )
{
    p.x = abs(p.x);
    float l = length(p) - r;
    float m = length(p-c*clamp(dot(p,c),0.0,r)); // c=sin/cos of aperture
    return max(l,m*sign(c.y*p.x-c.x*p.y));
}
```



Cut Disk - exact

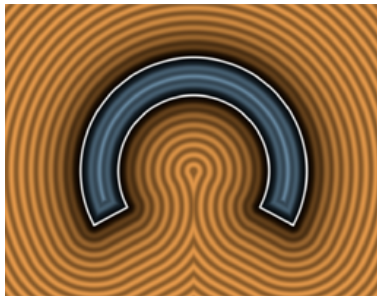
<https://www.shadertoy.com/view/ftVXRc>

```
float sdCutDisk( in vec2 p, in float r, in float h )
{
    float w = sqrt(r*r-h*h); // constant for any given shape
    p.x = abs(p.x);
    float s = max( (h-r)*p.x*p.x+w*w*(h+r-2.0*p.y), h*p.x-w*p.y );
    return (s<0.0) ? length(p)-r :
        (p.x<w) ? h - p.y :
            length(p-vec2(w,h));
}
```

**Arc - exact**

<https://www.shadertoy.com/view/wl23RK>

```
float sdArc( in vec2 p, in vec2 sc, in float ra, float rb )
{
    // sc is the sin/cos of the arc's aperture
    p.x = abs(p.x);
    return ((sc.y*p.x>sc.x*p.y) ? length(p-sc*ra) :
            abs(length(p)-ra)) - rb;
}
```

**Ring - exact**

<https://www.shadertoy.com/view/DsccDH>

```
float sdRing( in vec2 p, in vec2 n, in float r, float th )
{
    p.x = abs(p.x);
    p = mat2x2(n.x,n.y,-n.y,n.x)*p;
    return max( abs(length(p)-r)-th*0.5,
                length(vec2(p.x,max(0.0,abs(r-p.y)-th*0.5)))*sign(p.x) );
}
```

**Horseshoe - exact**

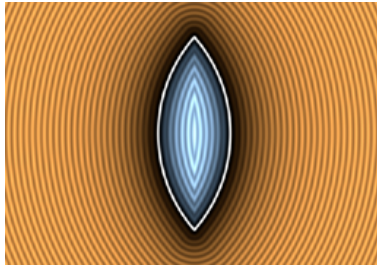
<https://www.shadertoy.com/view/WISGW1>

```
float sdHorseshoe( in vec2 p, in vec2 c, in float r, in vec2 w )
{
    p.x = abs(p.x);
    float l = length(p);
```

```

p = mat2(-c.x, c.y, c.y, c.x)*p;
p = vec2((p.y>0.0 || p.x>0.0)?p.x:1*sign(-c.x),
         (p.x>0.0)?p.y:1 );
p = vec2(p.x,abs(p.y-r))-w;
return length(max(p,0.0)) + min(0.0,max(p.x,p.y));
}

```



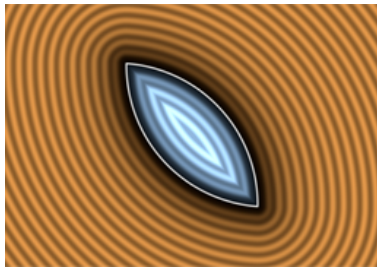
Vesica - exact

<https://www.shadertoy.com/view/XtVfRW>

```

float sdVesica(vec2 p, float w, float h)
{
    vec3 d = 0.5*(w*w-h*h)/h;
    p = abs(p);
    vec3 c = (w*p.y<d*(p.x-w)) ? vec3(0.0,w,0.0) : vec3(-d,0.0,d+h);
    return length(p-c.yx) - c.z;
}

```



Oriented Vesica - exact

<https://www.shadertoy.com/view/cs2yzG>

```

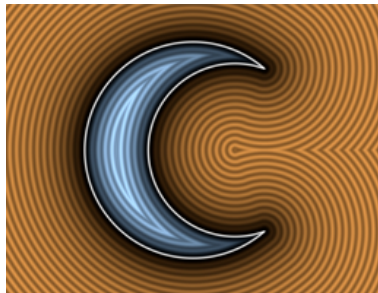
float sdOrientedVesica( vec2 p, vec2 a, vec2 b, float w )
{
    float r = 0.5*length(b-a);
    float d = 0.5*(r*r-w*w)/w;
    vec2 v = (b-a)/r;
    vec2 c = (b+a)*0.5;
    vec2 q = 0.5*abs(mat2(v.y,v.x,-v.x,v.y)*(p-c));
    vec3 h = (r*q.x<d*(q.y-r)) ? vec3(0.0,r,0.0) : vec3(-d,0.0,d+w);
}

```

```

return length( q-h.xy) - h.z;
}

```



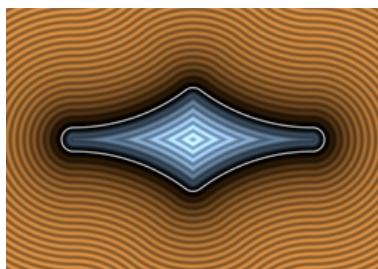
Moon - exact

<https://www.shadertoy.com/view/WtdBRS>

```

float sdMoon(vec2 p, float d, float ra, float rb )
{
    p.y = abs(p.y);
    float a = (ra*ra - rb*rb + d*d)/(2.0*d);
    float b = sqrt(max(ra*ra-a*a,0.0));
    if( d*(p.x*b-p.y*a) > d*d*max(b-p.y,0.0) )
        return length(p-vec2(a,b));
    return max( (length(p) -ra),
                -(length(p-vec2(d,0))-rb));
}

```



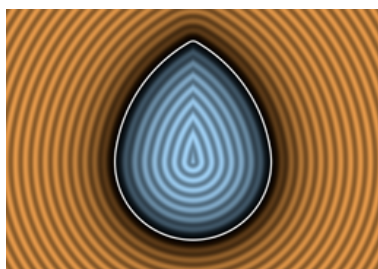
Circle Cross - exact

<https://www.shadertoy.com/view/NslXDM>

```

float sdRoundedCross( in vec2 p, in float h )
{
    float k = 0.5*(h+1.0/h);
    p = abs(p);
    return ( p.x<1.0 && p.y<p.x*(k-h)+h ) ?
        k-sqrt(dot2(p-vec2(1,k))) :
        sqrt(min(dot2(p-vec2(0,h)),
                dot2(p-vec2(1,0))));
}

```



Simple Egg - exact

<https://www.shadertoy.com/view/XtVfRW>

```

float sdEgg( in vec2 p, in float he, in float ra, in float rb, in float bu )
{
    // all this can be precomputed for any given shape
    float r = 0.5*(he + ra+rb)/bu;
    float da = r - ra;
    float db = r - rb;
}

```

```

float y = (db*db - da*da - he*he)/(2.0*he);
float x = sqrt(da*da - y*y);

// only this needs to be run per pixel
p.x = abs(p.x);
float k = p.y*x - p.x*y;
if( k>0.0 && k<he*(p.x+x) )
    return length(p+vec2(x,y))-r;
return min( length(p)-ra,
            length(vec2(p.x,p.y-he))-rb );
}

```



Heart - exact

<https://www.shadertoy.com/view/3tyBzV>

```

float sdHeart( in vec2 p )
{
    p.x = abs(p.x);

    if( p.y+p.x>1.0 )
        return sqrt(dot2(p-vec2(0.25,0.75))) - sqrt(2.0)/4.0;
    return sqrt(min(dot2(p-vec2(0.00,1.00)),
                    dot2(p-0.5*max(p.x+p.y,0.0)))) * sign(p.x-p.y);
}

```



Cross - exact exterior, bound interior

<https://www.shadertoy.com/view/XtGfzw>

```

float sdCross( in vec2 p, in vec2 b, float r )
{
    p = abs(p); p = (p.y>p.x) ? p.yx : p.xy;
    vec2 q = p - b;
    float k = max(q.y,q.x);
    vec2 w = (k>0.0) ? q : vec2(b.y-p.x,-k);
    return sign(k)*length(max(w,0.0)) + r;
}

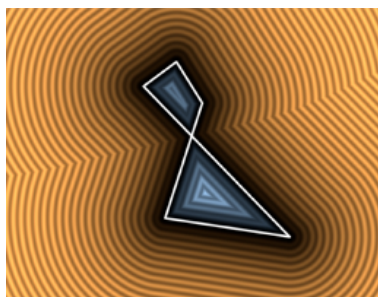
```



Rounded X - exact

<https://www.shadertoy.com/view/3dKSDc>

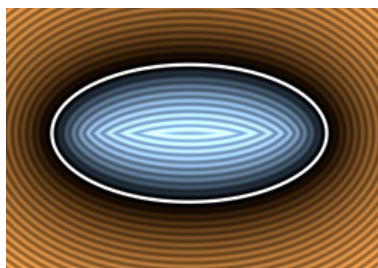
```
float sdRoundedX( in vec2 p, in float w, in float r )
{
    p = abs(p);
    return length(p-min(p.x+p.y,w)*0.5) - r;
}
```



Polygon - exact

<https://www.shadertoy.com/view/wdBXRW>

```
float sdPolygon( in vec2[N] v, in vec2 p )
{
    float d = dot(p-v[0],p-v[0]);
    float s = 1.0;
    for( int i=0, j=N-1; i<N; j=i, i++ )
    {
        vec2 e = v[j] - v[i];
        vec2 w = p - v[i];
        vec2 b = w - e*clamp( dot(w,e)/dot(e,e), 0.0, 1.0 );
        d = min( d, dot(b,b) );
        bvec3 c = bvec3(p.y>=v[i].y,p.y<v[j].y,e.x*w.y>e.y*w.x);
        if( all(c) || all(not(c)) ) s*=-1.0;
    }
    return s*sqrt(d);
}
```



Ellipse - exact

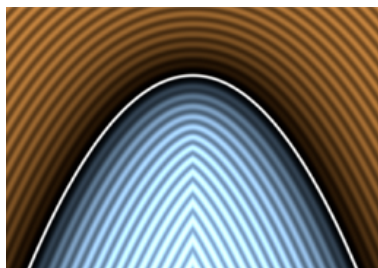
<https://www.shadertoy.com/view/4sS3zz>

```
float sdEllipse( in vec2 p, in vec2 ab )
{
    p = abs(p); if( p.x > p.y ) {p=p.yx;ab=ab.yx;}
    float l = ab.y*ab.y - ab.x*ab.x;
    float m = ab.x*p.x/l;          float m2 = m*m;
    float n = ab.y*p.y/l;          float n2 = n*n;
    float c = (m2+n2-1.0)/3.0; float c3 = c*c*c;
    float q = c3 + m2*n2*2.0;
```

```

float d = c3 + m2*n2;
float g = m + m*n2;
float co;
if( d<0.0 )
{
    float h = acos(q/c3)/3.0;
    float s = cos(h);
    float t = sin(h)*sqrt(3.0);
    float rx = sqrt( -c*(s + t + 2.0) + m2 );
    float ry = sqrt( -c*(s - t + 2.0) + m2 );
    co = (ry+sign(l)*rx+abs(g)/(rx*ry)- m)/2.0;
}
else
{
    float h = 2.0*m*n*sqrt( d );
    float s = sign(q+h)*pow(abs(q+h), 1.0/3.0);
    float u = sign(q-h)*pow(abs(q-h), 1.0/3.0);
    float rx = -s - u - c*4.0 + 2.0*m2;
    float ry = (s - u)*sqrt(3.0);
    float rm = sqrt( rx*rx + ry*ry );
    co = (ry/sqrt(rm-rx)+2.0*g/rm-m)/2.0;
}
vec2 r = ab * vec2(co, sqrt(1.0-co*co));
return length(r-p) * sign(p.y-r.y);
}

```



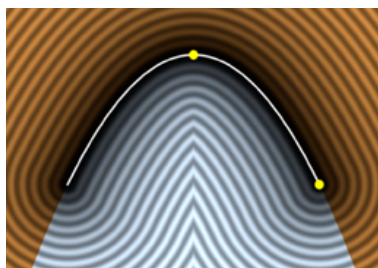
Parabola - exact

<https://www.shadertoy.com/view/ws3GD7>

```

float sdParabola( in vec2 pos, in float k )
{
    pos.x = abs(pos.x);
    float ik = 1.0/k;
    float p = ik*(pos.y - 0.5*ik)/3.0;
    float q = 0.25*ik*ik*pos.x;
    float h = q*q - p*p*p;
    float x;
    if( h>0.0 )
    {
        float r = pow(q+sqrt(h), 1.0/3.0);
        x = r + p/r;
    }
    else
    {
        float r = sqrt(p);
        x = 2.0*r*cos(acos(q/(p*r))/3.0);
    }
    return length(pos-vec2(x,k*x*x)) * sign(pos.x-x);
}

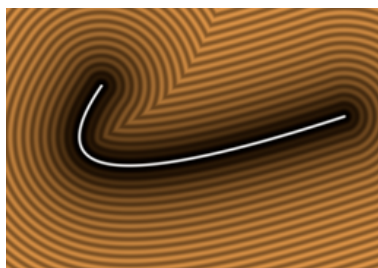
```



Parabola Segment - exact

<https://www.shadertoy.com/view/3lSczz>

```
float sdParabola( in vec2 pos, in float wi, in float he )
{
    pos.x = abs(pos.x);
    float ik = wi*wi/he;
    float p = ik*(he-pos.y-0.5*ik)/3.0;
    float q = pos.x*ik*ik/4.0;
    float h = q*q - p*p*p;
    float x;
    if( h>0.0 )
    {
        float r = pow(q+sqrt(h),1.0/3.0);
        x = r + p/r;
    }
    else
    {
        float r = sqrt(p);
        x = 2.0*r*cos(acos(q/(p*r))/3.0);
    }
    x = min(x,wi);
    return length(pos-vec2(x,he-x*x/ik)) *
        sign(ik*(pos.y-he)+pos.x*pos.x);
}
```



Quadratic Bezier - exact

<https://www.shadertoy.com/view/MIKcDD>

```
float sdBezier( in vec2 pos, in vec2 A, in vec2 B, in vec2 C )
{
    vec2 a = B - A;
    vec2 b = A - 2.0*B + C;
    vec2 c = a * 2.0;
    vec2 d = A - pos;
    float kk = 1.0/dot(b,b);
    float kx = kk * dot(a,b);
    float ky = kk * (2.0*dot(a,a)+dot(d,b)) / 3.0;
    float kz = kk * dot(d,a);
    float res = 0.0;
    float p = ky - kx*kx;
    float p3 = p*p*p;
}
```

```

float q = kx*(2.0*kx*kx-3.0*ky) + kz;
float h = q*q + 4.0*p3;
if( h >= 0.0)
{
    h = sqrt(h);
    vec2 x = (vec2(h,-h)-q)/2.0;
    vec2 uv = sign(x)*pow(abs(x), vec2(1.0/3.0));
    float t = clamp( uv.x+uv.y-kx, 0.0, 1.0 );
    res = dot2(d + (c + b*t)*t);
}
else
{
    float z = sqrt(-p);
    float v = acos( q/(p*z*2.0) ) / 3.0;
    float m = cos(v);
    float n = sin(v)*1.732050808;
    vec3 t = clamp(vec3(m+m,-n-m,n-m)*z-kx,0.0,1.0);
    res = min( dot2(d+(c+b*t.x)*t.x),
               dot2(d+(c+b*t.y)*t.y) );
    // the third root cannot be the closest
    // res = min(res,dot2(d+(c+b*t.z)*t.z));
}
return sqrt( res );
}

```



Bobbly Cross - exact

<https://www.shadertoy.com/view/NssXWM>

```

float sdBobblyCross( in vec2 pos, float he )
{
    pos = abs(pos);
    pos = vec2(abs(pos.x-pos.y), 1.0-pos.x-pos.y)/sqrt(2.0);

    float p = (he-pos.y-0.25/he)/(6.0*he);
    float q = pos.x/(he*he*16.0);
    float h = q*q - p*p*p;

    float x;
    if( h>0.0 ) { float r = sqrt(h); x = pow(q+r,1.0/3.0)-pow(abs(q-r),1.0/3.0)*sign(r-q); }
    else { float r = sqrt(p); x = 2.0*r*cos(acos(q/(p*r))/3.0); }
    x = min(x,sqrt(2.0)/2.0);

    vec2 z = vec2(x,he*(1.0-2.0*x*x)) - pos;
}

```

```

    return length(z) * sign(z.y);
}

```



Tunnel - exact

<https://www.shadertoy.com/view/flSSDy>

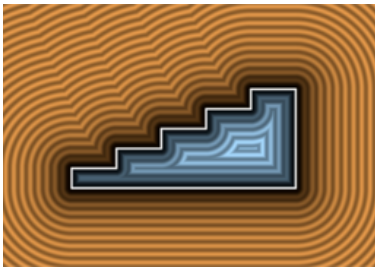
```

float sdTunnel( in vec2 p, in vec2 wh )
{
    p.x = abs(p.x); p.y = -p.y;
    vec2 q = p - wh;

    float d1 = dot2(vec2(max(q.x,0.0),q.y));
    q.x = (p.y>0.0) ? q.x : length(p)-wh.x;
    float d2 = dot2(vec2(q.x,max(q.y,0.0)));
    float d = sqrt( min(d1,d2) );

    return (max(q.x,q.y)<0.0) ? -d : d;
}

```



Stairs - exact

<https://www.shadertoy.com/view/7tKSWt>

```

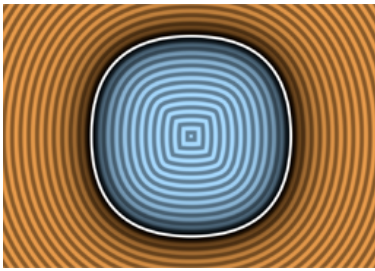
float sdStairs( in vec2 p, in vec2 wh, in float n )
{
    vec2 ba = wh*n;
    float d = min(dot2(p-vec2(clamp(p.x,0.0,ba.x),0.0)),
                  dot2(p-vec2(ba.x,clamp(p.y,0.0,ba.y)))) );
    float s = sign(max(-p.y,p.x-ba.x) );

    float dia = length(wh);
    p = mat2(wh.x,-wh.y, wh.y,wh.x)*p/dia;
    float id = clamp(round(p.x/dia),0.0,n-1.0);
    p.x = p.x - id*dia;
    p = mat2(wh.x, wh.y,-wh.y,wh.x)*p/dia;

    float hh = wh.y/2.0;
    p.y -= hh;
    if( p.y>hh*sign(p.x) ) s=1.0;
    p = (id<0.5 || p.x>0.0) ? p : -p;
    d = min( d, dot2(p-vec2(0.0,clamp(p.y,-hh,hh))) );
    d = min( d, dot2(p-vec2(clamp(p.x,0.0,wh.x),hh)) );

    return sqrt(d)*s;
}

```



Quadratic Circle - exact

<https://www.shadertoy.com/view/Nd3cW8>

```
float sdQuadraticCircle( in vec2 p )
{
    p = abs(p); if( p.y>p.x ) p=p.yx;

    float a = p.x-p.y;
    float b = p.x+p.y;
    float c = (2.0*b-1.0)/3.0;
    float h = a*a + c*c*c;
    float t;
    if( h>=0.0 )
    {
        h = sqrt(h);
        t = sign(h-a)*pow(abs(h-a),1.0/3.0) - pow(h+a,1.0/3.0);
    }
    else
    {
        float z = sqrt(-c);
        float v = acos(a/(c*z))/3.0;
        t = -z*(cos(v)+sin(v)*1.732050808);
    }
    t *= 0.5;
    vec2 w = vec2(-t,t) + 0.75 - t*t - p;
    return length(w) * sign( a*a*0.5+b-1.5 );
}
```



Hyperbola - exact

<https://www.shadertoy.com/view/DtjXDG>

```

float sdHyberbola( in vec2 p, in float k, in float he ) // k in (0,inf)
{
    p = abs(p);
    p = vec2(p.x-p.y,p.x+p.y)/sqrt(2.0);

    float x2 = p.x*p.x/16.0;
    float y2 = p.y*p.y/16.0;
    float r = k*(4.0*k - p.x*p.y)/12.0;
    float q = (x2 - y2)*k*k;
    float h = q*q + r*r*r;
    float u;
    if( h<0.0 )
    {
        float m = sqrt(-r);
        u = m*cos( acos(q/(r*m))/3.0 );
    }
    else
    {
        float m = pow(sqrt(h)-q,1.0/3.0);
        u = (m - r/m)/2.0;
    }
    float w = sqrt( u + x2 );
    float b = k*p.y - x2*p.x*2.0;
    float t = p.x/4.0 - w + sqrt( 2.0*x2 - u + b/w/4.0 );
    t = max(t,sqrt(he*he*0.5+k)-he/sqrt(2.0));
    float d = length( p-vec2(t,k/t) );
    return p.x*p.y < k ? d : -d;
}

```



Cool S - exact

<https://www.shadertoy.com/view/clVXWc>

```

float sdfCoolS( in vec2 p )
{
    float six = (p.y<0.0) ? -p.x : p.x;
    p.x = abs(p.x);
    p.y = abs(p.y) - 0.2;
    float rex = p.x - min(round(p.x/0.4),0.4);
    float aby = abs(p.y-0.2)-0.6;

    float d = dot2(vec2(six,-p.y)-clamp(0.5*(six-p.y),0.0,0.2));
    d = min(d,dot2(vec2(p.x,-aby)-clamp(0.5*(p.x-aby),0.0,0.4)));
    d = min(d,dot2(vec2(rex,p.y -clamp(p.y,0.0,0.4))));
}

```

```

float s = 2.0*p.x + aby + abs(aby+0.4) - 0.4;
return sqrt(d) * sign(s);
}

```



Circle Wave - exact

<https://www.shadertoy.com/view/stGyzt>

```

float sdCircleWave( in vec2 p, in float tb, in float ra )
{
    tb = 3.1415927*5.0/6.0*max(tb,0.0001);
    vec2 co = ra*vec2(sin(tb),cos(tb));
    p.x = abs(mod(p.x,co.x*4.0)-co.x*2.0);
    vec2 p1 = p;
    vec2 p2 = vec2(abs(p.x-2.0*co.x),-p.y+2.0*co.y);
    float d1 = ((co.y*p1.x>co.x*p1.y) ? length(p1-co) : abs(length(p1)-ra));
    float d2 = ((co.y*p2.x>co.x*p2.y) ? length(p2-co) : abs(length(p2)-ra));
    return min(d1, d2);
}

```

Making shapes rounded

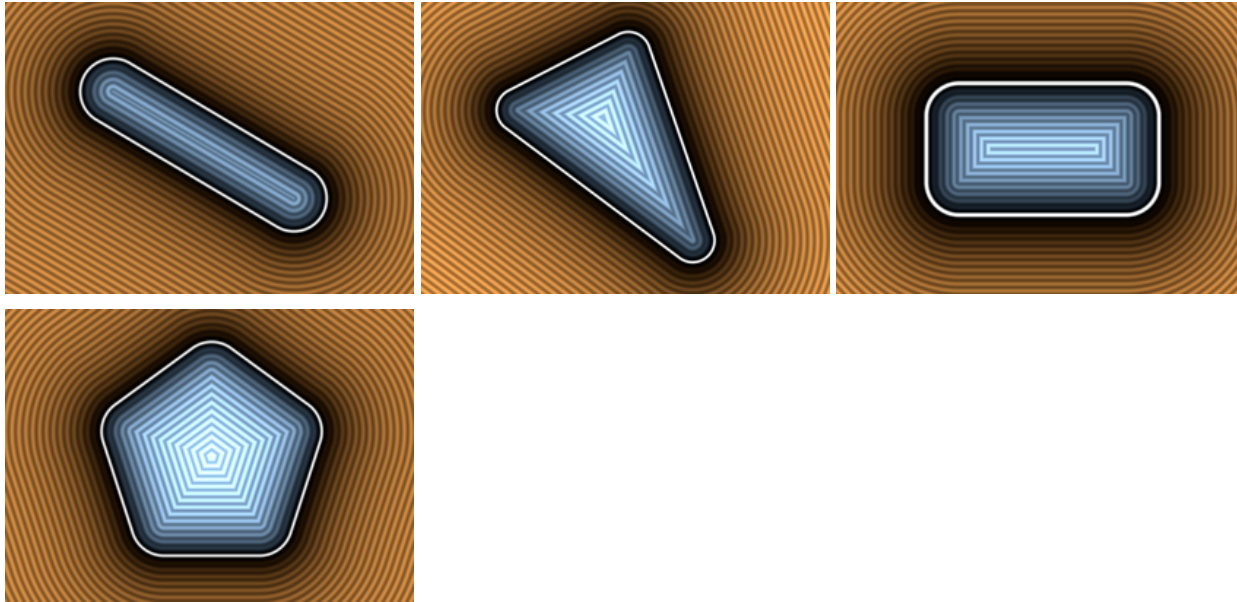
All the shapes above can be converted into rounded shapes by subtracting a constant from their distance function. That, effectively moves the isosurface (isoperimeter I guess) from the level zero to one of the outer rings, which naturally are rounded, as it can be seen in the yellow areas in all the images above. So, basically, for any shape defined by $d(x,y) = sdf(x,y)$, one can make it rounded by computing $d(x,y) = sdf(x,y) - r$. You can learn more about this in [this Youtube video](#).

```

float opRound( in vec2 p, in float r )
{
    return sdShape(p) - r;
}

```

These are a few examples: rounded line, rounded triangle, rounded box and a rounded pentagon:



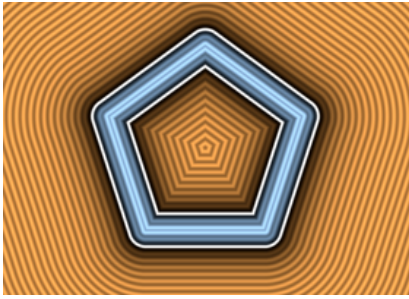
Making shapes annular

Similarly, shapes can be made annular (like a ring or the layers of an onion), but taking their absolute value and then subtracting a constant from their field. So, for any shape defined by $d(x,y) = \text{sdf}(x,y)$ compute $d(x,y) = |\text{sdf}(x,y)| - r$:

```
float opOnion( in vec2 p, in float r )
{
    return abs(sdShape(p)) - r;
}
```

These are a few examples: annular rounded line, an annular triangle, an annular box and a annular pentagon:





Rigid deformations, domain repetition, range distortion, boolean operations, smooth connections, etc

All of these work just as well as in 3D case, so I won't repeat the examples already exist in [this article](#).

[inigo quilez](#) - learning computer graphics since 1994